Demo Kit / Walkthrough

ChainCert

Blockchain-Based Credential Issuance and Verification Platform

March 26, 2026 — CAP-2459-IT

Table of Contents

1. System Overview

2. Important Notes and Pre-Demo Setup

3. Demo Accounts

4. Problem Proposal Mapping

Core Demonstration Scenarios

- Scenario 1: Issuer Registration
- Scenario 2: Root Authority Governance
- Scenario 3: Admin Approves Issuer
- Scenario 4: Issuer Creates Credential Template
- Scenario 5: Credential Issuance
- Scenario 6: Student Credential Wallet
- Scenario 7: Credential Verification
- Scenario 8: Credential Revocation
- Scenario 9: CouchDB vs Blockchain Integrity Check
- Scenario 10: Blockchain CLI Verification (Live Terminal)

Additional Feature Scenarios (If Time Permits)

- Scenario A1: Admin Governance Logs and Notifications
- Scenario A2: Credential Expiration
- Scenario A3: Profile Sharing
- Scenario A4: PDF Export
- Scenario A5: Visibility Toggle
- Scenario A6: Forgot Password / Reset Password

Appendix A: Blockchain Query Commands

1. System Overview

ChainCert is a Blockchain-Based Credential Issuance and Verification Platform designed to provide a secure and tamper-resistant system for issuing digital credentials.

Traditional credential systems rely on centralized databases that can be altered or forged. Verification often requires manual confirmation from issuing institutions, making the process slow and unreliable.

ChainCert addresses these issues by integrating Hyperledger Fabric blockchain technology with a modern web application architecture. Through this platform:

- Approved organizations can issue digital credentials
- Credential holders can manage their credentials in a digital wallet
- Third parties can verify credentials through the system
- Administrative governance ensures only validated issuers participate

Important credential lifecycle operations — including credential issuance, revocation, expiration, and verification logging — are recorded on an immutable blockchain ledger, ensuring transparency and trust.

On-Chain Data (Blockchain)

- Credential issuance and template creation records
- Credential revocation and expiration records
- Credential verification logs
- Issuer governance decisions (approval, rejection, enable, disable)
- Root authority subordinate decisions
- Administrative status changes and password change audits

Off-Chain Data (CouchDB)

- User accounts and issuer profiles
- Notifications (user and admin)
- Credential wallet views and display preferences

- Profile sharing tokens and OTP records
- Issuer onboarding documents (as CouchDB attachments)

2. Important Notes and Pre-Demo Setup

2.1 Blockchain Network

The system requires a running Hyperledger Fabric network with the ChainCert smart contract deployed.

- Channel: chaincert-channel
- Smart Contract: chaincert/index.js
- Orderer: orderer.example.com (port 7050) — single Raft orderer
- Peers: peer0.org1 (port 7051), peer0.org2 (port 9051), peer0.dlsu (port 12051)
- Certificate Authorities: ca_org1 (7054), ca_org2 (8054), ca_dlsu (12054), ca_orderer (9054)

In the current architecture, ChainCert manages all infrastructure centrally. Root authority organizations are assigned peer nodes and Fabric identities. Subordinate issuers operate using Fabric identities issued by the root authority's CA, without deploying their own peer infrastructure.

2.2 Security Mechanisms

- Passwords are hashed using bcrypt (bcryptjs) before storage
- The backend API uses JWT authentication on every request
- Fabric identities are secured using X.509 certificates issued by Fabric CA

2.3 Database Infrastructure

The system uses CouchDB for off-chain application data.

Port	Purpose	Databases
5985	Admin / System-wide database	users, issuers, user_notifications, admin_notifications, couchdb_connections, profile_share_links, wallet_user_credentials
9084	Root Authority Issuer (DLSU)	wallet_issuer_credentials, issued_credentials

9085	Subordinate Issuer (INNOCENCIO)	wallet_issuer_credentials, issued_credentials
9086	Subordinate Issuer (La Salle Computer Society)	wallet_issuer_credentials, issued_credentials

2.4 Demo Environment Setup

To reduce demonstration time, the blockchain network and participating organization nodes were configured prior to the demonstration. This includes: Fabric CA setup, peer node deployment, channel creation and joining, chaincode installation and approval, CouchDB container initialization, and Fabric identity enrollment.

> *The full procedure for deploying organization nodes is documented in the Installation Manual (CAP-2459-IT).*

3. Demo Accounts

The following accounts were created prior to the demonstration.

Administrator Accounts

Two administrator accounts are used to demonstrate admin audit logging — showing which admin performed which action.

Email	Password	Role
admin1@email.com	pw123	ChainCert Administrator
admin2@email.com	pw123	ChainCert Administrator

Root Authority Issuer

Email	Password	Role	Fabric Identity
dlsu@email.com	pw123	Issuer (Root Authority)	delasalleuniversity-admin

Subordinate Issuers

Email	Password	Role	Root Authority	Fabric Identity
lasallecomputersociety@email.com	pw123	Issuer (Subordinate)	De La Salle University	delasalleuniversity-subordinate-lasallecomputersociety

INNOCENCIO is also a configured subordinate issuer (port 9085, identity: delasalleuniversity-subordinate-innocencio) but La Salle Computer Society is used as the primary subordinate for this demonstration.

Credential Holder

Email	Password	Role	Full Name
kylejustin.tan7@gmail.com	pw123	Credential Holder	Kyle Justin Tan

Identity Summary

Identity Type	Example	Purpose
System Admin Account	admin1@email.com	Login to admin dashboard (CouchDB)
Fabric Admin Identity	chaincert-admin	Performs blockchain operations (system-level)
Issuer Fabric Identity	delasalleuniversity-admin	Root authority credential operations
Subordinate Identity	delasalleuniversity-subordinate-lasallecomputersociety	Subordinate credential operations
Subordinate Identity	delasalleuniversity-subordinate-innocencio	Subordinate credential operations

4. Problem Proposal Mapping

The following table maps each specific problem identified in the CAP-2459-IT proposal to the ChainCert features that address it. Scenarios are referenced so panelists can track which problems are being addressed during the demonstration.

#	Proposal Problem	How ChainCert Addresses It	Demo Scenario
1	Reliance on Static Credential Formats Without Embedded Verifiability	Credentials are blockchain-backed records with unique IDs that can be verified instantly through the verification interface — not static PDFs or paper documents that must be taken at face value.	Scenario 7
2	Lack of Credential Lifecycle Management (Issuance, Expiry, Revocation, and Updates)	Full lifecycle implemented: template creation, issuance with expiry dates, automated expiration via cron job, revocation with reason and holder notification — each stage recorded on blockchain with timestamps.	Scenarios 4, 5, 8, A2
3	Absence of Standardized Credential Metadata and Format Structures	Credential templates enforce a consistent structure (credential name, type, description, skills, badge image, metadata). All issued credentials inherit this structure regardless of which issuer creates them.	Scenario 4
4	Exclusion of Microcredentials and Informal Learning Proofs	The platform supports multiple issuer categories (universities, training providers, government agencies, industry certifiers) under one ecosystem. The unified wallet stores credentials from all issuer types — formal and informal — verifiable through one interface. This addresses the fragmentation where credentials from different issuers currently exist in separate, disconnected systems.	Scenarios 4, 6
5	Decentralized and Paper-Based Storage Results in Credential Loss and Inaccessibility	Credentials are stored in a digital wallet accessible anytime via the platform. Blockchain provides permanent backup — credentials cannot be lost even if the application database is compromised. Profile sharing enables controlled distribution without physical documents.	Scenarios 6, A3

6	Manual Verification Delays Due to Lack of Credential Interoperability	Verification is instant and requires no contact with the issuing institution. Any verifier can check any credential using just the credential ID — no account required, no waiting period, result backed by blockchain.	Scenario 7
7	No Tamper-Evident Logs or Version History	Every credential lifecycle event and governance decision is logged on the immutable blockchain with transaction IDs and timestamps. The credential history timeline provides a complete audit trail. The integrity check validates that off-chain records match the blockchain.	Scenarios 6, 9, 10
8	No Unified Registry or Searchable Credential Index	The admin dashboard provides a system-wide credential view across all issuers. The verification interface serves as a unified lookup point. The integrity check validates that the registry is synchronized with the blockchain source of truth.	Scenarios 9, A1
9	Credentials Are Not Interoperable Across Systems	Multiple issuer types and categories coexist on the same blockchain channel. Credentials from root authorities and subordinate issuers are verifiable through the same verification interface regardless of the issuing organization.	Scenarios 2, 7

Note: Problems 4, 8, and 9 are partially addressed within the current prototype scope. The system architecture supports future expansion for fuller cross-system interoperability.

Core Demonstration Scenarios

The following scenarios demonstrate the credential lifecycle within the ChainCert platform, ordered by priority. Each scenario builds on the previous one, forming a continuous narrative from issuer onboarding through credential verification and governance oversight.

> **Core scenarios are designed to fit within the 40-minute defense demonstration window. Additional feature scenarios follow after the core and may be demonstrated if time permits.**

Scenario 1: Issuer Registration

> **Context**
>
> *A new organization, La Salle Computer Society, wants to issue digital credentials to its members. Before they can do anything on the platform, they must first register as an issuer and submit their documents for review.*

Step 1 **[Issuer]** Access the Create Account page and select the Issuer Registration option.

📷 *Screenshot: Issuer Registration Page*

Step 2 **[Issuer]** Fill out the onboarding form including: organization name, contact information, issuer category, and issuer type (Root Authority or Subordinate Issuer). Since La Salle Computer Society is a subordinate, the system requires selection of De La Salle University as the root authority.

📷 *Screenshot: Issuer Registration Form*

Step 3 **[Issuer]** Upload the required verification documents. The system dynamically loads the required documents based on the selected issuer category.

📷 *Screenshot: Document Upload Section*

Step 4 **[Issuer]** Submit the application. The system records the issuer application with a status of Pending Approval.

📷 *Screenshot: Application Submitted Confirmation*

The issuer account cannot log in until the application has passed both the root authority approval gate and the platform administrator approval.

At this point, the La Salle Computer Society application is sitting in the system as "Pending." Because they registered as a subordinate under DLSU, their application must first be reviewed by the root authority before the platform admin can act.

Scenario 2: Root Authority Governance

Context

Because La Salle Computer Society registered as a subordinate issuer under De La Salle University, their application cannot go directly to the platform administrator. DLSU — the root authority — must review and approve the application first. This is the two-tier approval gate: root authority approves first, then admin can act.

Step 1 **[Root Authority]** Log in as DLSU (dlsu@email.com) and navigate to the Root Authority Governance section.

📷 *Screenshot: Root Authority Dashboard*

Step 2 **[Root Authority]** View pending subordinate applications. La Salle Computer Society's application appears in the list.

📷 *Screenshot: Pending Subordinate Applications*

Step 3 **[Root Authority]** Review the subordinate's details and submitted documents.

📷 *Screenshot: Subordinate Application Review*

Step 4 **[Root Authority]** Approve the subordinate issuer application. The decision is recorded on the blockchain via LogRootSubordinateApproval.

📷 *Screenshot: Subordinate Approval Action*

To Verify (Blockchain)

The root authority decision is recorded on-chain and can be viewed in the Root Authority Governance Logs.
Blockchain command: GetAllRootSubordinateLogs (see Appendix A)

DLSU has now approved La Salle Computer Society as a legitimate subordinate. The application's root approval status changes to "Root Approved," and a notification is sent to the platform administrator. However, the subordinate still cannot log in — the platform administrator must give final approval.

Scenario 3: Admin Approves Issuer

Context

Following DLSU's root approval, the platform administrator now sees the La Salle Computer Society application in the admin dashboard under root-approved subordinates awaiting admin action. The admin reviews the submitted documents and decides whether to grant full platform access.

Step 1 **[Admin]** Log in as admin1@email.com and open the Admin Dashboard.

📷 *Screenshot: Admin Dashboard*

Step 2 **[Admin]** Navigate to pending issuer applications. The La Salle Computer Society application appears with a "Root Approved" badge.

📷 *Screenshot: Pending Issuer Applications*

Step 3 **[Admin]** Review the uploaded documents to verify the legitimacy of the organization.

📷 *Screenshot: Document Viewer*

Step 4 **[Admin]** Approve the issuer. The system assigns a Fabric identity, resolves the CouchDB URL from couchdb_connections, and generates a wallet ID. The decision is recorded on the blockchain via LogIssuerApproval.

📷 *Screenshot: Issuer Approval Action*

For root authority issuers, the administrator must first deploy the organization's blockchain node manually (CA, peer, channel join, chaincode install) before clicking approve. For subordinate issuers, only the Fabric CA identity registration and wallet import are required — no separate peer node is deployed.

To Verify (Blockchain)

The approval decision is recorded on-chain and visible in the Admin Dashboard issuer decision logs.

Blockchain command: GetAllIssuerDecisionLogs (see Appendix A)

La Salle Computer Society is now a fully approved issuer. They can log in to the Issuer Dashboard and begin creating credential templates. The approval decision has been permanently recorded on the blockchain and cannot be altered or deleted.

— Credential Lifecycle Demonstration —

Scenarios 1–3 demonstrated the governance pipeline: issuer registration, root authority approval, and admin approval. Scenarios 4–8 demonstrate the credential lifecycle using De La Salle University (dlsu@email.com) — the root authority issuer. DLSU has full blockchain infrastructure deployed and provides richer demonstration data. The credential operations (template creation, issuance, revocation) are identical regardless of whether the issuer is a root authority or subordinate — the only difference is the Fabric identity used to sign transactions.

Scenario 4: Issuer Creates Credential Template

Context

De La Salle University, the root authority issuer, logs into the Issuer Dashboard to define a credential template — the blueprint that describes what the credential represents. Before any issuer can issue credentials, they must first create a template that enforces a consistent structure.

Step 1 **[Issuer]** Log in as dlsu@email.com and navigate to the Create Credential Template section.

📷 *Screenshot: Template Creation Page*

Step 2 **[Issuer]** Define the credential details including: credential name, credential type, description, metadata, and associated skills.

📷 *Screenshot: Template Form Fields*

Step 3 **[Issuer]** Upload a badge image representing the credential.

📷 *Screenshot: Badge Upload*

Step 4 **[Issuer]** Save the template. The template is recorded on both the blockchain (via CreateCredentialTemplate) and CouchDB (for fast dashboard access). This is the dual-write pattern — blockchain first, then CouchDB.

📷 *Screenshot: Template Created Successfully*

To Verify (Blockchain)

The template record can be viewed in the Issuer Dashboard credential history.

Blockchain command: Get_IssuerCredentialTemplates (see Appendix A)

The template has been saved. De La Salle University can now use this template to issue credentials to individual holders.

Scenario 5: Credential Issuance

Context

With the credential template ready, De La Salle University can now issue a credential to a specific holder. Kyle Justin Tan is a student at DLSU who has earned a credential. The issuer will issue the credential to Kyle's account.

Step 1 **[Issuer]** Select the credential template created in the previous scenario.

📷 *Screenshot: Template Selection*

Step 2 **[Issuer]** Select the recipient from the eligible users list. The system filters out users who already hold an active (non-revoked, non-expired) credential for this template, preventing duplicate issuance.

📷 *Screenshot: Recipient Selection*

Step 3 **[Issuer]** Configure optional parameters such as expiration date.

Step 4 **[Issuer]** Confirm issuance. The system submits an IssueCredential transaction to the blockchain, then saves copies to the issuer's CouchDB (issued_credentials) and the admin CouchDB (wallet_user_credentials). A notification is created for the holder.

📷 *Screenshot: Credential Issued*

To Verify (Blockchain)

The issuance record is visible in the Issuer Credential History page.

Blockchain command: Get_IssuerIssuedCredentials (see Appendix A)

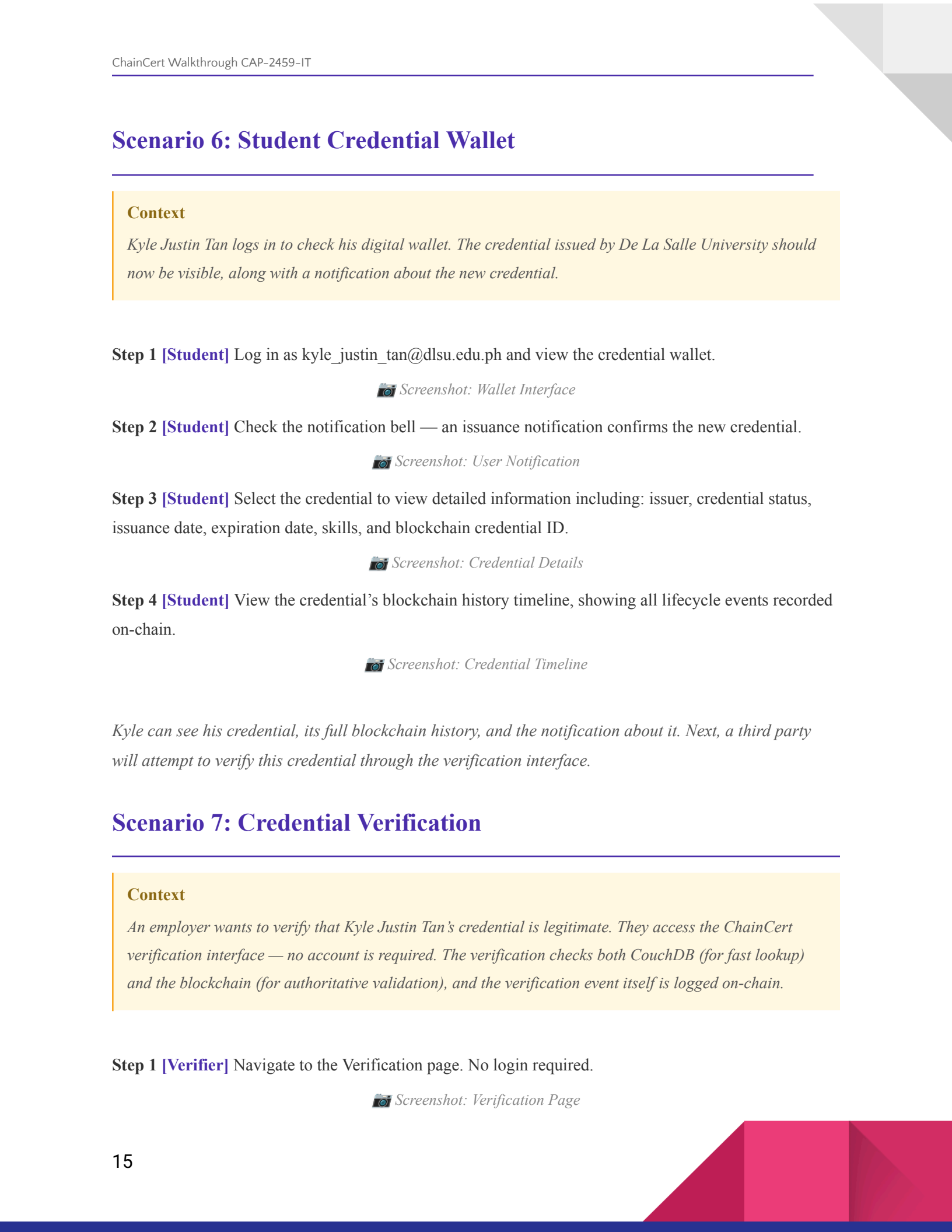
Kyle Justin Tan has now received a credential. It will appear in his digital wallet the next time he logs in, along with a notification informing him of the new credential.

Scenario 6: Student Credential Wallet

Context

Kyle Justin Tan logs in to check his digital wallet. The credential issued by De La Salle University should now be visible, along with a notification about the new credential.

Step 1 **[Student]** Log in as kyle_justin_tan@dlsu.edu.ph and view the credential wallet.


Screenshot: Wallet Interface

Step 2 **[Student]** Check the notification bell — an issuance notification confirms the new credential.

Screenshot: User Notification

Step 3 **[Student]** Select the credential to view detailed information including: issuer, credential status, issuance date, expiration date, skills, and blockchain credential ID.

Screenshot: Credential Details

Step 4 **[Student]** View the credential's blockchain history timeline, showing all lifecycle events recorded on-chain.

Screenshot: Credential Timeline

Kyle can see his credential, its full blockchain history, and the notification about it. Next, a third party will attempt to verify this credential through the verification interface.

Scenario 7: Credential Verification

Context

An employer wants to verify that Kyle Justin Tan's credential is legitimate. They access the ChainCert verification interface — no account is required. The verification checks both CouchDB (for fast lookup) and the blockchain (for authoritative validation), and the verification event itself is logged on-chain.

Step 1 **[Verifier]** Navigate to the Verification page. No login required.

Screenshot: Verification Page

Step 2 **[Verifier]** Provide verifier identity (name and email).

📷 *Screenshot: Verifier Form*

Step 3 **[Verifier]** Enter the credential ID (obtained from the credential details page, PDF, or shared link).

📷 *Screenshot: Verification Input*

Step 4 **[System]** The system checks the blockchain ledger via VerifyCredential to confirm credential validity (existence, revocation status, expiration status). The result is displayed to the verifier.

📷 *Screenshot: Verification Result*

Step 5 **[System]** The verification event is logged on the blockchain via LogVerification, creating an immutable audit trail of who verified what and when.

> **To Verify (Blockchain)**
>
> Verification logs are visible in the Admin Dashboard verification logs table.
>
> Blockchain command: GetAllVerificationLogs (see Appendix A)

The employer has confirmed the credential is valid. The verification event has been permanently logged. Next, we'll demonstrate what happens when an issuer revokes a credential.

Scenario 8: Credential Revocation

> **Context**
>
> *De La Salle University discovers that the credential was issued in error and needs to revoke it. The revocation will be recorded on the blockchain, the credential status will update across all CouchDB copies, and the holder will receive a revocation notification.*

Step 1 **[Issuer]** Log in as dlsu@email.com and navigate to the issued credentials list.

📷 *Screenshot: Issued Credential List*

Step 2 **[Issuer]** Select the credential to revoke and provide a revocation reason.

📷 *Screenshot: Revocation Modal*

Step 3 [Issuer] Submit the revocation. The system submits a RevokeCredential transaction to the blockchain, updates both CouchDB copies (issuer DB + admin wallet DB), and sends a revocation notification to the holder.

📷 *Screenshot: Revocation Confirmed*

Step 4 [Student] When Kyle logs in, the credential now shows as "Revoked" and a revocation notification appears.

📷 *Screenshot: Revoked Credential in Wallet*

Step 5 [Verifier] If the verifier re-checks the same credential ID, the system now returns "Revoked" with the revocation reason.

📷 *Screenshot: Verification Result — Revoked*

> **To Verify (Blockchain)**
>
> The revocation record is visible in the Issuer Credential History interface.
>
> Blockchain command: Get_CredentialTemplateOrIssuedCredentialHistory (see Appendix A)

The credential lifecycle is now complete: created, issued, verified, and revoked — all recorded immutably on the blockchain. Next, we'll demonstrate the integrity check feature that proves the blockchain backing is real.

Scenario 9: CouchDB vs Blockchain Integrity Check

> **Context**
>
> *To prove that the blockchain backing is not just cosmetic, the platform administrator triggers an integrity check. This feature compares every issued credential's CouchDB record against its blockchain record field by field — covering credentialName, credentialType, holderID, status, issuedDate, expiryDate, description, and issuerId. Any mismatch is flagged.*

Step 1 [Admin] Log in as admin2@email.com (demonstrating that a different admin account can perform this action — both admin actions are logged with the acting admin's email).

📷 *Screenshot: Admin Dashboard — Admin 2*

Step 2 **[Admin]** Navigate to the integrity check section and trigger the check.

📷 *Screenshot: Integrity Check Trigger*

Step 3 [System] The system iterates through all issued credentials, retrieves each record from both CouchDB and blockchain, and compares them field by field.

Step 4 **[Admin]** Review the integrity check results. A clean result shows all records match.

📷 *Screenshot: Integrity Check Results — Clean*

Live Tamper Demonstration

Step 5 **[Admin]** Open the CouchDB Fauxton admin UI at localhost:9084 (DLSU's CouchDB instance) and navigate to the issued_credentials database.

📷 *Screenshot: Fauxton — DLSU issued_credentials*

Step 6 **[Admin]** Find a credential document and manually modify a field — for example, change the credentialName or status field. Save the change in Fauxton.

📷 *Screenshot: Fauxton — Modified Record*

Step 7 **[Admin]** Return to the Admin Dashboard and re-run the integrity check.

📷 *Screenshot: Integrity Check Trigger — Re-run*

Step 8 **[Admin]** The integrity check now flags the tampered credential with the specific mismatched fields highlighted.

📷 *Screenshot: Integrity Check Results — Mismatch Detected*

This proves that CouchDB is synchronized with the blockchain, and any tampering is detectable. The final core scenario demonstrates direct blockchain querying.

Scenario 10: Blockchain CLI Verification (Live Terminal)

Context

To demonstrate that on-chain data is independently verifiable beyond the application interface, the administrator opens a terminal and queries the blockchain directly using Fabric CLI commands. This proves that the data is on the actual ledger, not just in the application's database.

Step 1 **[Admin]** Open a terminal and configure the Fabric CLI environment (see Appendix A, Prerequisites).

Step 2 **[Admin]** Query a specific credential by ID to show the on-chain record.

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredential","CREDENTIAL_ID"]}'
```

Step 3 **[Admin]** Query the credential's full blockchain history to show every lifecycle event (creation, issuance, revocation).

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredentialHistory","CREDENTIAL_ID"]}'
```

Step 4 **[Admin]** Query verification logs to show that the verification event from Scenario 7 is recorded.

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllVerificationLogs"]}'
```

Step 5 **[Admin]** Query issuer decision logs to show the approval from Scenario 3 is on-chain.

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllIssuerDecisionLogs"]}'
```

> *For the full list of available blockchain query commands, refer to Appendix A at the end of this document, or the Technical Manual (CAP-2459-IT) for detailed command reference with example values and expected outputs.*

Additional Feature Scenarios

Context

The following scenarios cover additional features of the ChainCert platform beyond the core credential lifecycle. These may be demonstrated if time permits during the defense presentation.

Scenario A1: Admin Governance Logs and Notifications

Context

Administrators have access to comprehensive audit logs and a notification system that keeps them informed of platform events in real time.

Step 1 **[Admin]** View the admin notification bell. Notifications are triggered by: issuer registration, root subordinate decisions, and admin approval/rejection/disable actions.

📷 *Screenshot: Admin Notifications*

Step 2 **[Admin]** Mark notifications as read (individually or all at once).

Step 3 **[Admin]** Navigate to the audit logs section. Available logs include: issuer status logs (enable/disable), user status logs (enable/disable), issuer decision logs (approval/rejection with admin email attribution), root subordinate decision logs, and admin password change logs.

📷 *Screenshot: Admin Audit Logs Dashboard*

Step 4 **[Admin]** Demonstrate admin-level attribution by comparing actions performed by admin1@email.com vs admin2@email.com in the logs. This proves that even though both admins share the same Fabric identity (chaincert-admin), individual accountability is preserved through logged metadata.

Step 5 **[Admin]** Trigger a manual credential expiry check to demonstrate the admin's ability to process expired credentials on demand.

📷 *Screenshot: Manual Expiry Check*

Step 6 **[Admin]** Enable or disable a user account to show the admin's governance control. The status change is logged on the blockchain.

📷 *Screenshot: User Enable/Disable*

Scenario A2: Credential Expiration

> **Context**
>
> *Credentials with expiration dates are automatically processed by a midnight cron job. The admin can also trigger the check manually. When a credential expires, its status is updated in CouchDB and recorded on the blockchain via ExpireCredential.*

Step 1 **[Issuer]** Issue a credential with a past or near-past expiration date (for demo purposes).

Step 2 **[Admin]** Trigger the manual expiry check from the Admin Dashboard.

Step 3 [System] The system identifies expired credentials, updates CouchDB, and logs each expiration on the blockchain.

Step 4 **[Student]** The credential now shows as "Expired" in the student's wallet.

Step 5 **[Verifier]** Verifying the expired credential now returns "Expired" with the expiration timestamp.

Scenario A3: Profile Sharing

> **Context**
>
> *Credential holders can share their credential profile with third parties through shareable links. The system supports two modes: public links (no authentication required) and recipient-specific links (requiring email + OTP verification).*

Step 1 **[Student]** Navigate to the Profile Sharing section and generate a share link. Choose between: Public Link (anyone with the link can view) or Recipient-Specific Link (requires recipient's email for OTP verification).

📷 *Screenshot: Share Link Generation*

Step 2 **[Student]** View and manage active share links. Links can be revoked at any time.

📷 *Screenshot: My Share Links*

Step 3 **[Verifier]** Access the shared link. For recipient-specific links, the system sends a 6-digit OTP to the recipient's email.

📷 *Screenshot: OTP Verification Page*

Step 4 **[Verifier]** After OTP verification (or directly for public links), the viewer sees only the credentials marked as public by the holder.

📷 *Screenshot: Public Profile Page*

Scenario A4: PDF Export

> **Context**
>
> *Credential holders can export their credentials as PDF documents for offline sharing.*

Step 1 **[Student]** From the credential details page, click Export PDF. The system supports single credential export and portfolio/transcript export (all credentials).

📷 *Screenshot: PDF Export Options*

Step 2 **[Student]** The exported PDF includes: holder information, credential details, blockchain credential ID, issuer information, and a verification link.

📷 *Screenshot: Exported PDF*

Scenario A5: Visibility Toggle

> **Context**
>
> *Holders can control which credentials appear on their shared profile, and issuers can control which templates are publicly visible.*

Step 1 **[Student]** Toggle a credential between Public and Private. Private credentials do not appear on shared profile links.

📷 *Screenshot: Credential Visibility Toggle*

Step 2 **[Issuer]** Toggle a credential template between Public and Private. This controls template visibility on the issuer's shared profile (only templates, not issued credentials).

📷 *Screenshot: Template Visibility Toggle*

Scenario A6: Forgot Password / Reset Password

> **Context**
>
> *Users who forget their password can reset it through an OTP-based email verification flow.*

Step 1 **[Student]** Click “Forgot Password” on the login page and enter the registered email.

📷 *Screenshot: Forgot Password Page*

Step 2 [System] A time-limited 6-digit OTP is sent to the email address.

Step 3 **[Student]** Enter the OTP and set a new password. The system rejects the new password if it is the same as the current password.

📷 *Screenshot: Reset Password Page*

Appendix A: Blockchain Query Commands

The following commands may be used to verify that credential lifecycle events and governance actions are recorded on the Hyperledger Fabric blockchain ledger. These commands interact directly with the chaincert chaincode deployed on the chaincert-channel.

> *For full command reference with actual example values and expected outputs, refer to the Technical Manual (CAP-2459-IT), Appendix A.*

1. Prerequisites

Run these commands once per terminal session before executing any queries.

```
cd ~/fabric-samples/fabric-samples/test-network

export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=${PWD}/../config/

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="DLSUMSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/dlsu.example.com/peers/peer0.dlsu.exa
mple.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/dlsu.example.com/users/Admin@dlsu.exam
ple.com/msp
export CORE_PEER_ADDRESS=localhost:12051
```

2. Credential Records

Get a Specific Credential

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredential","CREDENTIAL_ID"]}'
```

Get Credential History

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_CredentialTemplateOrIssuedCredentialHistory","CREDENTIAL_ID"]}'
```

Check Credential Exists

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Check_CredentialExists","CREDENTIAL_ID"]}'
```

Verify Credential Validity

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["VerifyCredential","CREDENTIAL_ID"]}'
```

3. Issuer Credential Records

Get Templates by Issuer

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_IssuerCredentialTemplates","ISSUER_ID"]}'
```

Get Issued Credentials by Issuer

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_IssuerIssuedCredentials","ISSUER_ID"]}'
```

Get Issuer Credential History

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_IssuerCredentialsHistory","ISSUER_ID"]}'
```

4. Holder Credentials

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["Get_HolderCredentials","HOLDER_ID"]}'
```

5. Verification Logs

Get All Verification Logs

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllVerificationLogs"]}'
```

Get Specific Verification Log

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetVerificationLogById","VERIFICATION_LOG_ID"]}'
```

6. Governance and Administrative Logs

Issuer Decision Logs (All)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllIssuerDecisionLogs"]}'
```

Issuer Decision Logs (by Email)

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetIssuerDecisionLogs","ISSUER_EMAIL"]}'
```

Issuer Status Change Logs

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllIssuerStatusLogs"]}'
```

User Status Change Logs

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllUserStatusLogs"]}'
```

Root Authority Subordinate Logs

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetAllRootSubordinateLogs"]}'
```

Root Subordinate Logs by Root Authority

```
peer chaincode query \
  -C chaincert-channel \
  -n chaincert \
  -c '{"Args":["GetRootSubordinateLogsByRootAuthority","ROOT_AUTHORITY_ID"]}'
```

7. Administrative Security Logs

Admin Password Change Logs

```
peer chaincode query \
```

```
-C chaincert-channel \
-n chaincert \
-c '{"Args":["GetAdminPasswordChangeLogs","ADMIN_EMAIL"]}'
```